

Nom du lab et du repos : **k8s-observability-helm-stack**

Lab — Prometheus + Grafana sur Kubernetes (Step-by-Step propre)

Sommaire

Lab — Prometheus + Grafana sur Kubernetes (Step-by-Step propre)	1
1 Objectifs.....	2
Step 0 — Prérequis (incluant Helm).....	2
 0.1 Kubernetes.....	2
 0.2 Installer Helm.....	2
 Vérification finale.....	3
Step 1 — Installer Prometheus + Grafana (Helm).....	3
1.1 Ajouter le repo.....	3
1.2 Namespace dédié et lancer cluster.....	3
1.3 Installation de Helm.....	3
Step 2 — Inspection immédiate.....	4
Inspecter un service	4
Logs	5
Step 3 — Accès Grafana.....	5
Mot de passe	5
Port-forward	5
Step 4 — Explorer Grafana.....	6
Step 5 — Accès Prometheus.....	6
Test	7
On peut lister tous les pods et endpoints avec Prometheus. En cliquant sur l’onglet status puis sur “target health”	7
La figure ci-dessous montre par exemple le pod : alertmanager-monitoring-kube-prometheus-alertmanager-0.....	7
On voit l’état de chaque pods : ip ou down.....	7
Step 6 — Déployer une app.....	8
Step 7 — Service.....	9

Step 8 — ServiceMonitor.....	10
Step 9 — Vérification.....	11
9.1 : Accéder à Prometheus.....	11
9.2 vérifier les targets 🖱️ Status → Targets.....	11
9.3 Autre moyen de faire : avec des queries.....	11
9.4 Résultat.....	11
Step 10 — Conclusion.....	12
10.1 Limitation :	12
10.2 Amélioration :	12

1 Objectifs

- Installer **Helm**
- Déployer **Prometheus + Grafana**
- Comprendre le **scraping**
- Visualiser les métriques
- Appliquer une démarche **debug / inspection**

Step 0 — Prérequis (incluant Helm)

🔧 0.1 Kubernetes

Cluster local : minikube **ou** k3d
kubectl get nodes

🔧 0.2 Installer Helm

- 🖱️ Vérifie d'abord : helm version
- 🖱️ **Si Helm n'est PAS installé :**

Linux (WSL / Ubuntu)

```
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
```

Vérification finale

helm version
kubectl version --client

```
henri@Henri:~/labs/k8s-observability-helm-stack$ helm version
version.BuildInfo{Version:"v3.20.2", GitCommit:"8fb76d6ab555577e98e23b7500009537a471feee", GitTreeState:"clean", GoVersion:"go1.25.9"}
henri@Henri:~/labs/k8s-observability-helm-stack$ kubectl version --client
Client Version: v1.35.3
Kustomize Version: v5.7.1
henri@Henri:~/labs/k8s-observability-helm-stack$
```

Step 1 — Installer Prometheus + Grafana (Helm)

1.1 Ajouter le repo

helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update

1.2 Namespace dédié et lancer cluster

k3d cluster create henri

k3d cluster list

```
henri@Henri:~/labs/k8s-observability-helm-stack$ k3d cluster list
NAME          SERVERS  AGENTS  LOADBALANCER
gitops-lab    0/1      0/0      true
henri         1/1      0/0      true
ingress-lab   0/1      0/0      true
henri@Henri:~/labs/k8s-observability-helm-stack$
```

kubectl create namespace monitoring

puis kubectl get namespaces

```
henri@Henri:~/labs/k8s-observability-helm-stack$ kubectl get namespace
NAME                STATUS  AGE
default             Active  2m39s
kube-node-lease     Active  2m39s
kube-public         Active  2m40s
kube-system         Active  2m40s
monitoring          Active  62s
henri@Henri:~/labs/k8s-observability-helm-stack$
```

1.3 Installation de Helm

helm install monitoring prometheus-community/kube-prometheus-stack \

--namespace monitoring

Step 2 — Inspection immédiate

kubect! get all -n monitoring

kubect! get svc -n monitoring

👉 On dois voir :

- prometheus
- grafana
- alertmanager

```
henri@Henri:~/labs/k8s-observability-helm-stack$  
henri@Henri:~/labs/k8s-observability-helm-stack$ kubect! get all -n monitoring  
NAME                                READY   STATUS    RESTARTS   AGE  
pod/alertmanager-monitoring-kube-prometheus-alertmanager-0  2/2     Running   0           26s  
pod/monitoring-grafana-5d9d6dcf85-8p95h  2/3     Running   0           27s  
pod/monitoring-kube-prometheus-operator-77d477b5cb-mwt42  1/1     Running   0           27s  
pod/monitoring-kube-state-metrics-57fb56f896-fbdh6  1/1     Running   0           27s  
pod/monitoring-prometheus-node-exporter-mmfv8  1/1     Running   0           27s  
pod/prometheus-monitoring-kube-prometheus-prometheus-0  2/2     Running   0           26s
```

```
henri@Henri:~/labs/k8s-observability-helm-stack$ kubect! get svc -n monitoring  
NAME                                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)                                AGE  
alertmanager-operated               ClusterIP    None           <none>         9093/TCP,9094/TCP,9094/UDP            107s  
monitoring-grafana                  ClusterIP    10.43.112.129  <none>         80/TCP                                109s  
monitoring-kube-prometheus-alertmanager  ClusterIP    10.43.11.209   <none>         9093/TCP,8080/TCP                    109s  
monitoring-kube-prometheus-operator  ClusterIP    10.43.155.55   <none>         443/TCP                                109s  
monitoring-kube-prometheus-prometheus  ClusterIP    10.43.191.172  <none>         9090/TCP,8080/TCP                    109s  
monitoring-kube-state-metrics         ClusterIP    10.43.227.55   <none>         8080/TCP                                109s  
monitoring-prometheus-node-exporter  ClusterIP    10.43.105.211  <none>         9100/TCP                                109s  
prometheus-operated                 ClusterIP    None           <none>         9090/TCP                                107s
```

Inspecter un service

kubect! describe svc monitoring-grafana -n monitoring

- type ?
- ports ?
- endpoints ?

```
henri@Henri:~/labs/k8s-observability-helm-stack$
henri@Henri:~/labs/k8s-observability-helm-stack$ kubectl describe svc monitoring-grafana -n monitoring
Name: monitoring-grafana
Namespace: monitoring
Labels: app.kubernetes.io/instance=monitoring
        app.kubernetes.io/managed-by=Helm
        app.kubernetes.io/name=grafana
        app.kubernetes.io/version=13.0.1
        helm.sh/chart=grafana-12.1.3
Annotations: meta.helm.sh/release-name: monitoring
              meta.helm.sh/release-namespace: monitoring
Selector: app.kubernetes.io/instance=monitoring,app.kubernetes.io/name=grafana
Type: ClusterIP
IP Family Policy: SingleStack
IP Families: IPv4
IP: 10.43.112.129
IPs: 10.43.112.129
Port: http-web 80/TCP
TargetPort: grafana/TCP
Endpoints: 10.42.0.10:3000
Session Affinity: None
Internal Traffic Policy: Cluster
Events: <none>
henri@Henri:~/labs/k8s-observability-helm-stack$
```

Logs

On affiche tous les pods du namespace : monitoring ;

`kubectl get pods -n monitoring`

```
henri@Henri:~/labs/k8s-observability-helm-stack$ kubectl get pods -n monitoring
NAME                                                    READY   STATUS    RESTARTS   AGE
alertmanager-monitoring-kube-prometheus-alertmanager-0 2/2     Running   0           6m40s
monitoring-grafana-5d9d6dcf85-8p95h                    3/3     Running   0           6m41s
monitoring-kube-prometheus-operator-77d477b5cb-mwt42    1/1     Running   0           6m41s
monitoring-kube-state-metrics-57fb56f896-fbdh6          1/1     Running   0           6m41s
monitoring-prometheus-node-exporter-mmfv8              1/1     Running   0           6m41s
prometheus-monitoring-kube-prometheus-prometheus-0     2/2     Running   0           6m40s
henri@Henri:~/labs/k8s-observability-helm-stack$
```

`kubectl logs -n monitoring <pod-name>`

Step 3 — Accès Grafana

Mot de passe

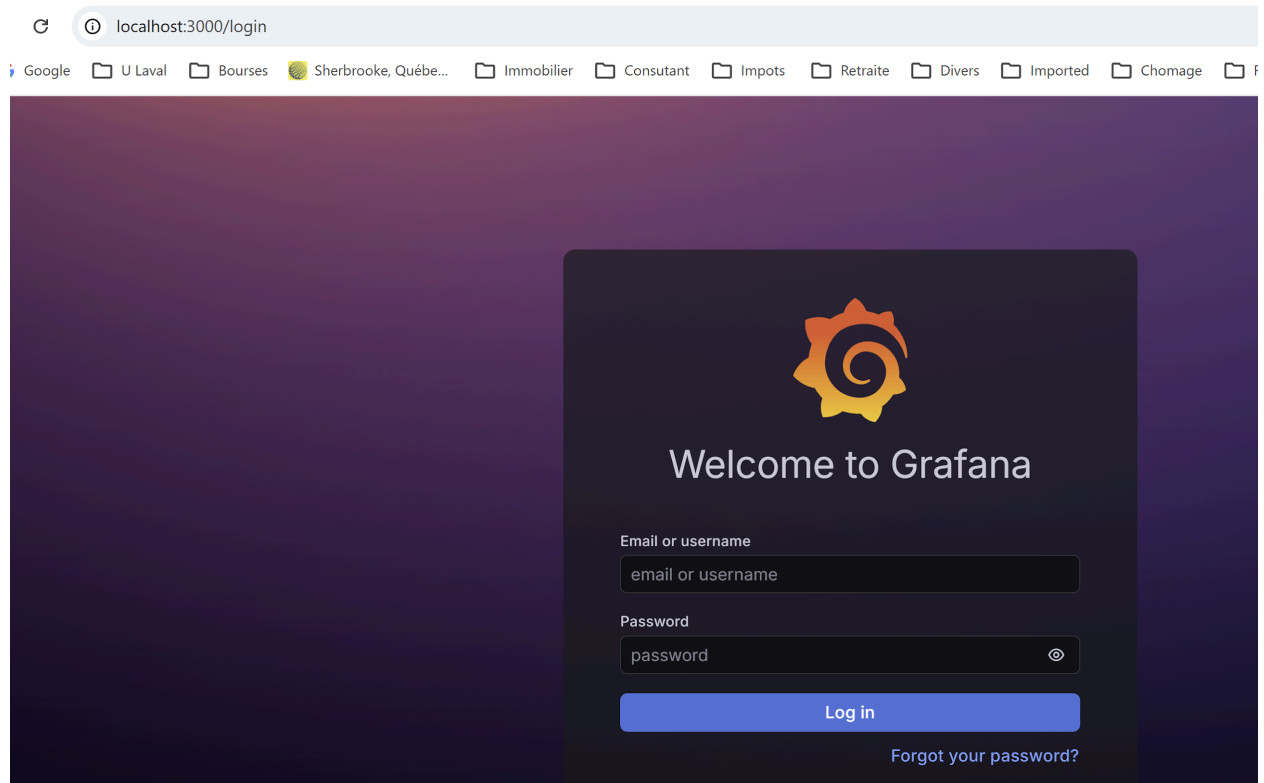
`kubectl get secret monitoring-grafana \`
`-n monitoring -o jsonpath="{.data.admin-password}" | base64 --decode`

Port-forward

```
kubectl port-forward svc/monitoring-grafana 3001:80 -n monitoring
```

👉 <http://localhost:3001>

login: admin



Step 4 — Explorer Grafana

- Dashboards → Browse
- Kubernetes / Nodes / Pods

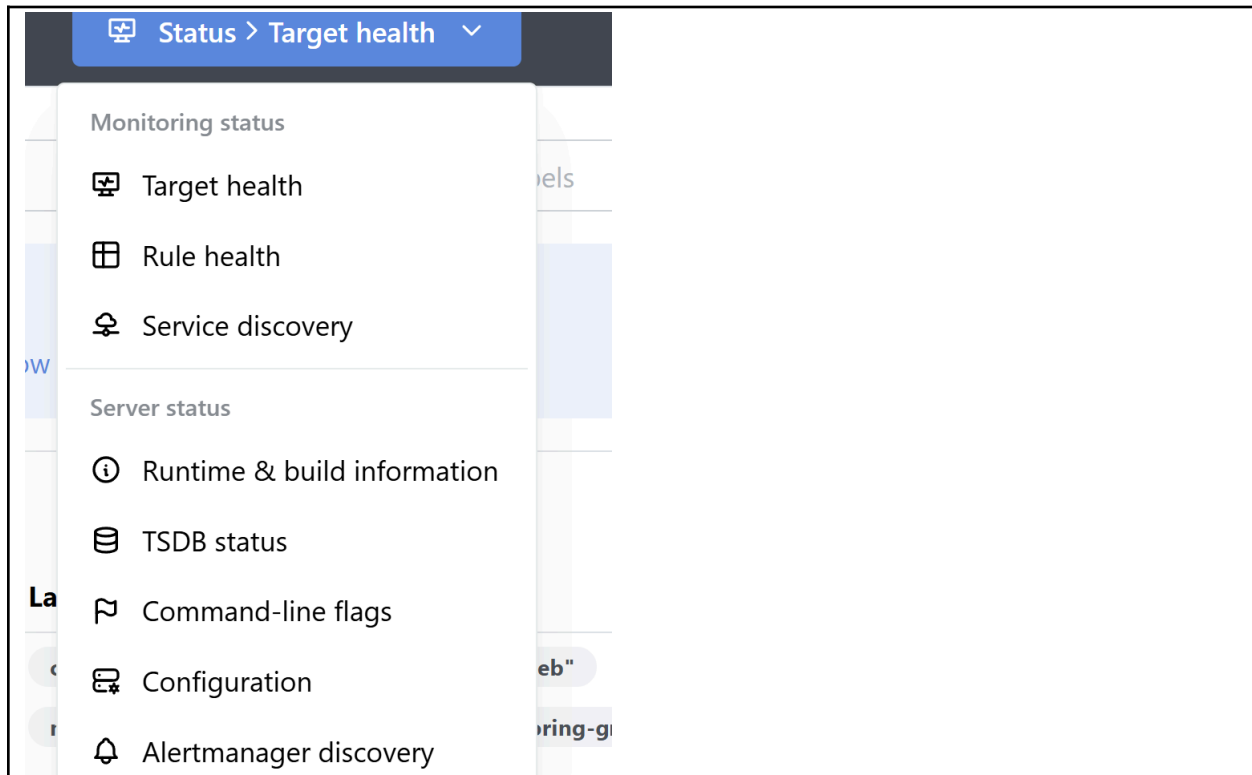
Step 5 — Accès Prometheus

```
kubectl port-forward svc/monitoring-kube-prometheus-prometheus 9090 -n monitoring
```

👉 <http://localhost:9090>

Test

On peut lister tous les pods et endpoints avec Prometheus. En cliquant sur l'onglet status puis sur "target health"



La figure ci-dessous montre par exemple le pod :
alertmanager-monitoring-kube-prometheus-alertmanager-0.

Les autres pods du cluster sont listés dessous.

On voit l'état de chaque pods : ip ou down

Step 6 — Déployer une app

Créer un fichier : app.yaml
Insérer le code ci-dessous

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: demo-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: demo-app
  template:
    metadata:
      labels:
        app: demo-app
    spec:
      containers:
        - name: demo-app
          image: nginx
          ports:
            - containerPort: 80
```

Puis appliquer :


```
kubectl apply -f app.yaml
```

Ensuite vérifier :
kubectl get pods

```
henri@Henri:~/labs/k8s-observability-helm-stack$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
demo-app-58bf5cfc8d-6wmcn          1/1     Running   0           5m47s
henri@Henri:~/labs/k8s-observability-helm-stack$
henri@Henri:~/labs/k8s-observability-helm-stack$
```

Le pod a été créé

Nb : on aurait pu créer le pod dans le namespace intitulé : monitoring avec la commande :
Kubectl apply -f app.yaml -n monitoring

Puis vérifier avec la commande :
Kubectl get pods -n monitoring

Step 7 — Service

Créer un fichier : service.yaml
Insérer le code ci-dessous

```
apiVersion: v1
kind: Service
metadata:
  name: demo-app
  labels:
    app: demo-app
spec:
  selector:
    app: demo-app
  ports:
    - name: http # ⚠ IMPORTANT
      port: 80
      targetPort: 80
```

Appliquer
kubectl apply -f service.yaml

vérifier :

kubectl get svc

```
henri@Henri:~/labs/k8s-observability-helm-stack$ kubectl get svc
NAME         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
demo-app     ClusterIP   10.43.53.53   <none>        80/TCP     22s
kubernetes   ClusterIP   10.43.0.1     <none>        443/TCP    18h
henri@Henri:~/labs/k8s-observability-helm-stack$
henri@Henri:~/labs/k8s-observability-helm-stack$
```

ou

kubectl get svc -n monitoring

inspecter :

kubectl describe svc demo-app

On doit voir :

selector → app=demo-app

endpoints → IP du pod

⚠ Si endpoints = vide
problème : labels mismatch
vérifier : kubectl get pods --show-labels

Step 8 — ServiceMonitor

Créer un fichier : servicemonitor.yaml

Insérer le code ci-dessous :

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: demo-app
  labels:
    release: monitoring
spec:
  selector:
    matchLabels:
      app: demo-app
  endpoints:
    - port: http
      interval: 15s
```

Appliquer :

```
kubectl apply -f servicemonitor.yaml
```

Vérifier :

```
kubectl get servicemonitors -n monitoring
```

```
henri@Henri:~/labs/k8s-observability-helm-stack$ kubectl get servicemonitors -n monitoring
NAME                                         AGE
monitoring-grafana                         18h
monitoring-kube-prometheus-alertmanager   18h
monitoring-kube-prometheus-apiserver      18h
monitoring-kube-prometheus-coredns        18h
monitoring-kube-prometheus-kube-controller-manager 18h
monitoring-kube-prometheus-kube-etcd      18h
monitoring-kube-prometheus-kube-proxy     18h
monitoring-kube-prometheus-kube-scheduler 18h
monitoring-kube-prometheus-kubelet        18h
monitoring-kube-prometheus-operator        18h
monitoring-kube-prometheus-prometheus     18h
monitoring-kube-state-metrics              18h
monitoring-prometheus-node-exporter        18h
henri@Henri:~/labs/k8s-observability-helm-stack$
```

peut lister tous les pods et endpoints avec Pr...

Step 9 — Vérification

Dans Prometheus :

9.1 : Accéder à Prometheus

```
kubectl port-forward svc/monitoring-kube-prometheus-prometheus 9090 -n monitoring
```

👉 <http://localhost:9090>

9.2 vérifier les targets 👉 Status → Targets

On devrait avoir :

- 🎬 job = demo-app
- 🎬 state = UP (ou DOWN si nginx sans metrics)

9.3 Autre moyen de faire : avec des queries

Dans Prometheus : query

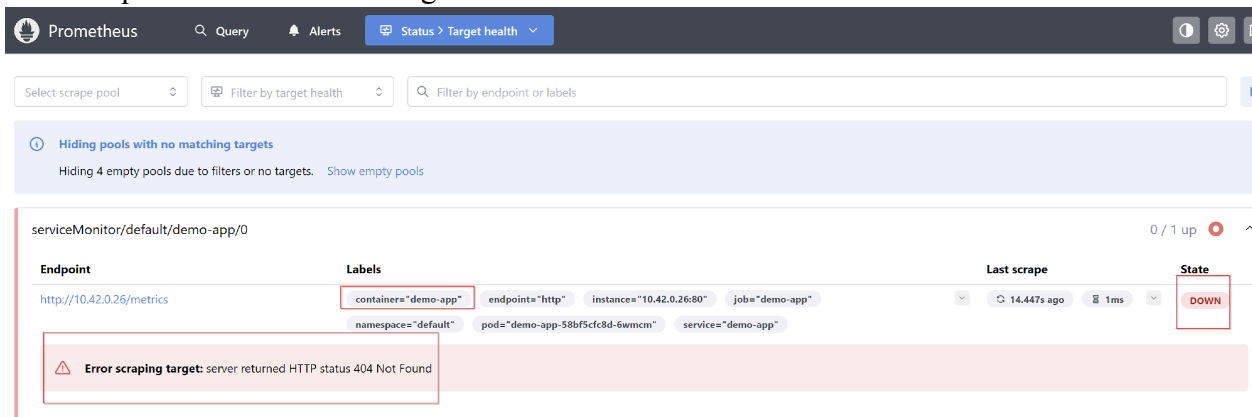
```
up{job="demo-app"}
```

9.4 Résultat

En fait l'application n'est pas monitorée. Elle est détectée par Prometheus et est affichée. Le container : demo-app est bien listée, mais il apparait comme down par l'utilitaire .

C'est en fait normal, car l'application containerisée est une application nginx, qui n'expose PAS /metrics. Et donc on a : `http://IP:80/metrics → 404`.

C'est ce qui est montré dans la figure ci-dessous :



Une autre manière de vérifier est de lancer la commande qui permet directement d'exposer l'application app-demo via le port 8080 et http

```
kubectl port-forward svc/demo-app 8080:80
```

puis :

<http://localhost:8080/metrics>

On obtient comme resultat : 404 not found

Ce qui est normal pour nginx, car elle ne donne pas accès /metrics

La solution serais d'introduire une exportation pour permettre aux métriques d'être vues par Prometheus. Voir le step 10 ci-dessous.

Step 10 — Conclusion

10.1 Limitation :

L'application de démonstration utilisée (Nginx) n'est pas instrumentée par défaut et n'expose pas de métriques Prometheus via un endpoint /metrics.

Par conséquent, Prometheus ne peut pas récupérer de métriques et les tentatives de scraping retournent une erreur HTTP 404.

Ce comportement est attendu.

10.2 Amélioration :

Pour corriger cela, plusieurs solutions sont possibles :

- Ajouter un exporter (ex : Nginx Prometheus Exporter) afin de transformer les données internes de Nginx en métriques exploitables par Prometheus
- Utiliser une application déjà instrumentée exposant un endpoint /metrics

- Instrumenter une application custom (Python, Go, etc.) pour exposer des métriques natives